# Exception / Error Handling

→ Error




1. avoid shutting down the full code
2. we should handle error gracefully.

## Exception / Error Handling

Python ~ Hindi, English

What is an error?

If we write code then we will face have errors:

1. Syntax error → During Compilation
2. Exceptions/ Runtime → During execution

## Syntax error

A syntax error occurs when python interpreter finds something wrong with the structure of the code.

→ not following rules of language.

→ program fails to run and will show message explaining the error.

- → parantheis
- → undefined variable
- → indendation
- → : (colon)

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[21], line 5
      1 # Index Error
      3 l1 = [1,2,3,4]
---> 5 l1[100]

IndexError: list index out of range
```

↑

python explaining us the error

We have to make sure that we correct and follow the rules of the language.

## Exceptions / Runtime Error

Exception is the error that occur during the execution of the program.

It doesn't prevent program from running but cause it to crash if not handeled. Normally faced when something unforeseen happens.

→ raised by python runtime.

Examples : division by Zero
Database Connection
Memory Overflow

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[49], line 4
      1 username = "testUser"
      2 password = "abcd" # 1234
----> 4 if(int(password)==1234):
      5     print("welcome to the System")

ValueError: invalid literal for int() with base 10: 'abcd'
```

### Stack Trace

1. very bad with respect to security.

→ application down
→ user experience
→ Security

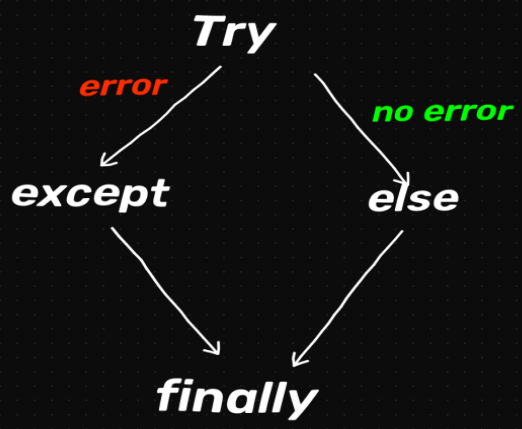
## Handling Exception in python

Try

Except

Else

Finally

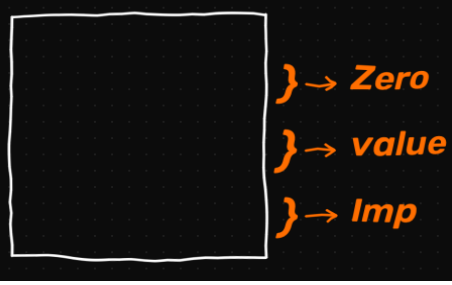



## Try

→ the 'try' block contains the code which might throw an exception/error

→ if an exception occur in this block, it is passed to the except block for handling.

Why not put everything inside try ?




We have to make sure that diff. errors are handled differently.

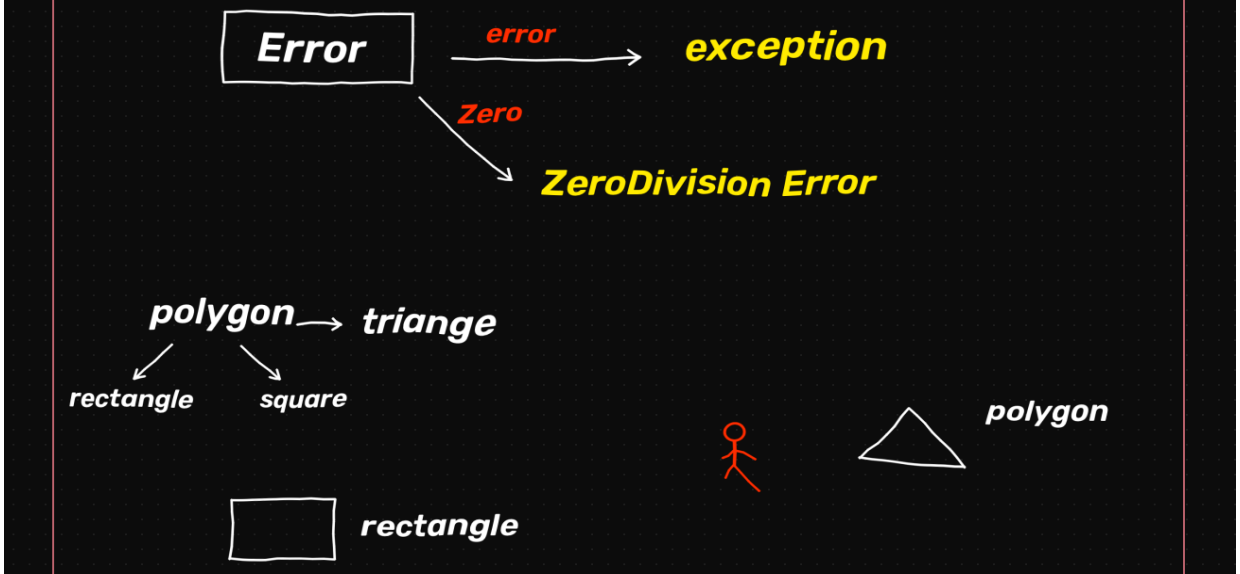
## Except

→ the 'except' block is used to catch and handle the exception that occurs in the try block.

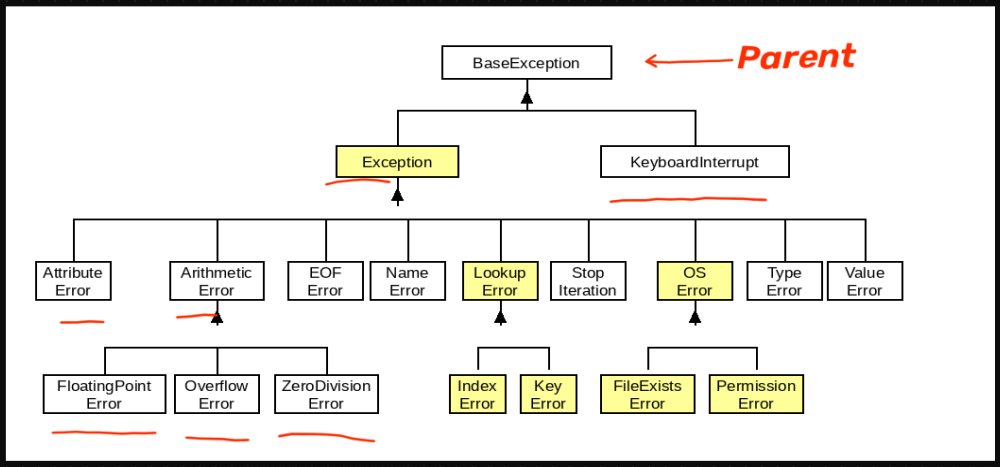
→ if an exception occurs ⇒ exception block gets executed

→ we can have multiple except blocks for different types of exception.

→ Order still matters



## Exception Tree in python




## Else

- the 'else' block gets executed when the 'try' block does not raise an exception
- It is a good place to put code that should run if no exceptions are raised.

## Raise

→ Python allows us to manually raise exception using the raise keyword

→ Useful when we want to enforce certain condition in our program.

## Custom error

→ Python allows us to create our own 'Custom Exception' by creating a new class.

→ allows us to create meaningful error messages specific to application / user need.

## Finally

→ the 'Finally' block is always executed, whether an exception is raised or not.

→ It is often used for cleanup action like 'closing files' or 'releasing resources'.

## You can create your own exceptions

1. Class
2. Inherit exception class